In Class Lab 5e

Keys That Make Themselves, and Triggers That Watch | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 90 minutes	lab5e_starter.sql	Your GitHub repository URL

Reference: https://www.cruzjohal.com/database/chapter-5-lab-guide

The due date is listed on Canvas LMS.

What this lab adds to Lab 5d

In Lab 5d you met AUTO_INCREMENT and wrote two triggers. This lab goes one layer deeper into both: how the server actually manages those generated key values, and what a trigger can do beyond copying a number from one table to another.

Section	What you add	Steps
5-7	Choose where a generated key starts, and see what DELETE and TRUNCATE do to the counter	4 to 7
5-8	A BEFORE trigger that changes the row on its way in, and an AFTER DELETE trigger	8 to 10

Surrogate keys, and why they exist

A **natural key** is a column the real world already gives you: a phone number, an email address, a course code. A **surrogate key** is a number the database invents purely to identify the row. student_id is a surrogate key. Nobody is born with one.

Surrogate keys are used so heavily because natural keys keep letting you down. People change their name and their email. Two students share a phone number. A course gets renumbered. Every one of those breaks a key that other tables are pointing at. A number the database made up has none of those problems, because it means nothing to anybody and so nobody ever needs to change it.

	Natural key	Surrogate key
Where it comes from	The real world	The server generates it
Example	An email address	student_id 1, 2, 3
Does it mean anything	Yes, which is the problem	No, which is the point
Can it change	Yes, and then every row pointing at it is wrong	There is never a reason to change it

Two things the counter does that surprise people

The server keeps one counter per table, and that counter is not the same thing as the number of rows. Lab 5d showed you the first of these. Step 6 shows you the second.

What you do	What the counter does
DELETE a row	Nothing. The number that row used is gone for good and the next insert takes the following one
DELETE every row	Still nothing. An empty table can have a counter sitting at 500
TRUNCATE the table	Resets it to the start. TRUNCATE is not a tidier DELETE, it throws the table away and makes a new empty one

Part 1. Get set up

Same routine as Lab 5d. SQLTools will not connect to a schema until one exists, so you connect to one that is always there and use it to make your own. You do not need the command prompt at any point in this lab.

Step 1. Connect to information_schema and make your folder

Make a folder on OneDrive called **CIS4720_Lab5e** through the OneDrive app, download **lab5e_starter.sql** from Canvas into it, and open that folder in VS Code with **File**, then **Open Folder**.

Then open the SQLTools panel, choose **Add New Connection** and pick **MySQL**. Server **localhost**, port **3306**, username **root**, your password, and for the database use **information_schema**. It is on every MySQL server, so this connection cannot fail. Test it, then save it.

Step 2. Create the schema, then point yourself at it

Open lab5e_starter.sql. The first two statements in it are the same two you ran in Lab 5d. Run them one at a time:

```
show databases;

create database if not exists cis_students_db;
```

The first lists every schema on the server. The second makes yours. Run **show databases;** again and **cis_students_db** should now be in the list.

You are still connected to information_schema, so the next line in the file switches you over:

```
use cis_students_db;
```

Run it, then add a second SQLTools connection with **Database** set to **cis_students_db** and work from that one for the rest of the lab. Some tools open a fresh connection for each statement and a fresh connection forgets the USE, so having the connection itself point at the right schema is what makes it reliable.

> **Check where you are before you build anything**
>
> Run **SELECT DATABASE();** at any point. It names the schema your statements are landing in right now. If it says information_schema or NULL when you expect cis_students_db, fix that before writing anything else.

Step 3. Run the rest of the starter file

Work down the file one statement at a time. The function near the bottom is a single statement from CREATE FUNCTION down to END, so select that whole block and run the selection. Then check you landed where you should:

Check	What you should see
SELECT * FROM students	Five rows, with gpa 85.00, 84.00, 92.33, 82.50 and 0.00
SELECT * FROM student_scores	Eleven rows, the last one Rita with NULL

> **No DELIMITER anywhere in this lab**
>
> Same as Lab 5d. The function and the triggers are each one statement, however many semicolons they contain. Select the whole block and run the selection.
>
> If you copy a trigger from the textbook or a web page it will probably be wrapped in **DELIMITER $$** and **DELIMITER ;** with **$$** after END. Delete all of that. It is an instruction to the command prompt program, not SQL, and the server rejects it with **error 1064**.

Part 2. Section 5-7, generated key values

Step 4. Look at the counters the server is keeping

The counter is not stored in your table. It lives in the server's own catalogue, and you can read it. This query is given to you because the catalogue is not somewhere you have been yet:

```
SELECT table_name, auto_increment
FROM information_schema.tables
WHERE table_schema = 'cis_students_db'
  AND auto_increment IS NOT NULL;
```

Two rows come back:

table_name	auto_increment	and your table has
students	7	5 rows
test_scores	11	10 rows

Neither number matches the row count, and neither is wrong. students says 7 because James Toor was row 6 and got deleted in Lab 5d. The counter moved past 6 and will never go back. Keep this query handy, because you run it again in Step 7.

Step 5. Create the attendance table, starting its key at 500

Real systems often start a generated key somewhere other than 1, so an id tells you at a glance which table it came from. Create a table named **attendance** with these four columns:

Column	Type	Rules
attendance_id	INT	AUTO_INCREMENT, and the primary key
student_id	INT	Never empty. A foreign key to student_id in students
attended_on	DATE	May be left empty
status	VARCHAR(10)	May be left empty

Then make the first generated id be **500** rather than 1. You can do that two ways and either is fine: add **AUTO_INCREMENT = 500** to the end of the CREATE TABLE statement, after the closing bracket and before the semicolon, or create the table normally and then use ALTER TABLE to set it. Look one of them up and use

it.

Describe the table afterwards and check attendance_id shows **PRI** and says **auto_increment** in the Extra column.

Step 6. Add three attendance rows and watch the ids

Insert these three rows. Name the columns you are supplying, so you do not have to pass anything for attendance_id:

student_id	attended_on	status
1	2026-10-05	present
2	2026-10-05	absent
3	2026-10-06	present

Then select the table. The three ids should read **500**, **501** and **502**, none of which you typed.

Step 7. What DELETE and TRUNCATE do to the counter

Do this on a throwaway table so your real data is never at risk. Create a table called **scrap** with just two columns, an AUTO_INCREMENT primary key called id and a VARCHAR(10) called label. Put three rows in it.

Then run the Step 4 counter query three times, doing one thing in between each, and write down what the counter says each time:

After you	The counter for scrap reads	Because
Insert three rows	4	Three ids used, so the next one is 4
DELETE FROM scrap	Still 4	Deleting rows does not give their numbers back
TRUNCATE TABLE scrap	1	The table was thrown away and remade empty

The Step 4 query only lists tables whose counter is not empty, so add **AND table_name = 'scrap'** to it if the output gets busy. Drop the scrap table when you are done.

> **TRUNCATE is not a tidier DELETE**
>
> DELETE removes rows one at a time and a trigger on the table sees every one of them. TRUNCATE throws the whole table away and builds an empty one with the same shape, which is why it resets the counter and why no DELETE trigger ever fires for it.
>
> It also refuses to run at all on a table that a foreign key points at. Try it on students and you get **ERROR 1701, cannot truncate a table referenced in a foreign key constraint**, which is the server protecting test_scores from being orphaned.

Part 3. Section 5-8, two more kinds of trigger

Both triggers in Lab 5d were AFTER triggers that updated another table. These two are different: the first changes the row on its way in, and the second reacts to a row going away.

Step 8. A BEFORE INSERT trigger that fills in what you left out

A **BEFORE** trigger runs while the row is still on its way to the table, which means it can change what gets stored. An AFTER trigger cannot, because by then the row is already written.

This one does two jobs: it puts today's date on any attendance row that arrived without one, and it forces the status into capitals so the column stays tidy. Copy it as it is.

```
-- runs while the row is still on its way in, so it can change it
CREATE TRIGGER trg_attendance_before_insert
BEFORE INSERT ON attendance
FOR EACH ROW
BEGIN
    -- no date supplied, so use today's
    IF NEW.attended_on IS NULL THEN
        SET NEW.attended_on = CURDATE();
    END IF;

    -- store the status in capitals whatever case it arrived in
    SET NEW.status = UPPER(NEW.status);
END
```

Three new pieces of procedural SQL

SET NEW.column = ... changes the row before it lands. This only works in a BEFORE trigger. Try it in an AFTER trigger and the server refuses, because the row has already been written and there is nothing left to change.

IF ... THEN ... END IF; is a decision. Everything between THEN and END IF only runs when the condition is true. Note that END IF has its own semicolon.

CURDATE() is a built in function giving today's date, so what you see depends on the day you run the lab.

Step 9. An AFTER DELETE trigger, the gap Lab 5d left open

Lab 5d finished with a problem on purpose. Your two triggers fired on INSERT and on UPDATE, so deleting a score left the gpa sitting at a number that was no longer true. This closes it.

```
-- a deleted score changes the average just as much as a new one
CREATE TRIGGER trg_scores_after_delete
AFTER DELETE ON test_scores
FOR EACH ROW
BEGIN
    -- OLD is the row that has just gone. there is no NEW here,
    -- because nothing was written
    UPDATE students
    SET gpa = calculate_gpa(OLD.student_id)
    WHERE student_id = OLD.student_id;
END
```

NEW and OLD, and when each one exists

This is the rule worth memorising, because reaching for the wrong one is the most common trigger mistake:

Trigger on	NEW	OLD
INSERT	The row being written	Does not exist
UPDATE	The values going in	The values being replaced
DELETE	Does not exist	The row that has just gone

Step 10. Prove both of them work

Two small tests. You write both, and you should be able to say what you expect before you run them.

Test the BEFORE trigger. Insert one more attendance row for student 4, naming only student_id and status, with the status in lower case as **present**. Supply no date at all. Then select the table.

What you supplied	What got stored
No date	Today's date, whatever today is
present	PRESENT
No attendance_id	503

Test the AFTER DELETE trigger. Jenna is student 1 and her gpa is 85.00. Add a quiz4 score of 100 for her, recalculate her gpa by hand the way Lab 5d did in its Step 9, then delete that quiz4 row and look at her gpa again without touching it yourself.

Jenna's gpa	After
85.00	you add the 100 and recalculate by hand
90.00	you delete the quiz4 row, trigger fires

That second line is the whole lab in one row. You deleted a score and the gpa corrected itself, with no UPDATE written by you anywhere.

Part 4. Push to GitHub

Step 11. Publish to a new repository

Save lab5e_starter.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize**.

1. Click **Source Control**, then **Publish to GitHub**.
2. Name it **cis4720-lab5e**. It has to be a name you have not used.
3. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
4. Tick **lab5e_starter.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.

> **Sign out of GitHub before you leave the lab**
>
> Shared PC. Your sign in stays on it until somebody removes it. Click **Accounts**, then your username, then **Sign Out**.

Step 12. Check it and submit the URL

1. Open the repository on github.com, click the file and read it on the page. What is there is exactly what gets graded.
2. Open the URL in a private or incognito window. A **404** means it is private, so go to **Settings** and change the visibility to public.
3. Paste the URL into the Lab 5e submission box on Canvas.

Submit the URL even if you did not finish. Partial work your instructor can read scores better than finished work nobody can reach.

How it is graded

Steps	What is being checked	Points
1 to 3	Connected, cis_students_db created, and the starter file running	1.0
4	The counter query run and the two values read	1.0
5	attendance created with its key starting at 500	2.0
6	Three rows in, with ids 500, 501 and 502	1.0
7	The scrap table, and the counter read after DELETE and after TRUNCATE	1.5
8	The BEFORE INSERT trigger created	1.0
9	The AFTER DELETE trigger created	1.0
10	Both triggers shown working	0.5
All	Required. A comment in your own words above every statement	1.0
11 to 12	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1064 near '--something' in a trigger or function	The space after the two dashes went missing in the paste. MySQL only treats -- as a comment when a space or a tab follows it, so --no is read as two minus signs. Put the space back, or use # instead, which needs no space.
ERROR 1064 near 'DELIMITER $$'	You copied a trigger that still has the DELIMITER lines around it. Delete them and the $$ after END.
ERROR 1064 on the SET or IF line	Your editor split the trigger at a semicolon inside it. Select the whole block from CREATE TRIGGER to END and run the selection.
ERROR 1362, updating of NEW row is not allowed	You put SET NEW.something in an AFTER trigger. Only a BEFORE trigger can change the row.
ERROR 1363, there is no NEW row in on delete trigger	Step 9 needs OLD, not NEW. A deleted row has no NEW.
ERROR 1701, cannot truncate a table referenced in a foreign key	You tried to TRUNCATE students. Use the scrap table for Step 7.
ERROR 1452, cannot add or update a child row	An attendance row names a student_id that is not in students.

What you see	What it means
ERROR 1359, trigger already exists	You ran Step 8 or 9 twice. Run DROP TRIGGER and the name, then create it again.
The status stayed lower case	The rows from Step 6 went in before the trigger existed. Only rows inserted afterwards are changed by it.

Stuck on syntax? The course lab guide works through all of it: https://www.cruzjohal.com/database/chapter-5-lab-guide